

yadiggg: Intelligence Graph and Schema Specification

Date of Document: Wednesday, June 3, 2026 **Time of Document:** 04:08 AM EDT **Status:** Master System Design (V2.0 - Core Graph & Database Schema) **Authors:** yadiggg System Architecture & Accio Product Architecture Suite

1. The Core Philosophy of Crate Credits

Unlike mainstream streaming platforms that organize music strictly by **Artist -> Album -> Song** to drive high-volume playlist plays, **yadiggg** organizes music by **Lineage, People, Connections, and Influence**.

For a vinyl collector or hip-hop producer, a song is not an isolated audio file—it is a node in a massive, historic network of collaboration. The magic of "crate digging" lies in discovering connections:

- *Who played bass on this obscure 1974 soul record?*
- *What other records did that same bass player touch?*
- *Who sampled this drum break, and where was it recorded?*

To answer these questions offline in sub-1.5 seconds on an ARM Cortex-A53 device, we design **The Intelligence Graph**. This normalized, relational database architecture is optimized specifically for physical record matching, deep credit exploration, and sample-lineage mapping.



2. The 5 Killer Screens: Interaction and Data Mapping

Every physical interaction (an optical label scan or an acoustic mic recording) maps to one of these five core displays.

Screen 1: Facts (The Foundation)

- **Purpose:** The landing screen for any successful scan. It mimics the physical spine or back jacket of a record, displaying standard release facts.
- **Key Fields Rendered:** Release Title, Artist, Year, Label, Country, Catalog Number, and Barcode.
- **Graph Source:** Queries `releases` and the primary `contributors` linked as the release artist.

Screen 2: Credits (The Collaboration)

- **Purpose:** The "Tidal-style" deep dive showing the session musicians, producers, and engineers who actually played on the track.

- **Key Fields Rendered:** Producer, Recording Engineer, Mix Engineer, Session Players (Bass, Drums, Vocals, Horns, Keys).
- **Graph Source:** Performs a join across `tracks` , `credits` , `contributors` , and `roles` for the active track.

Screen 3: Sample Tree (The Lineage)

- **Purpose:** The ultimate hip-hop tool. It shows bidirectional lineage—what samples this track contains, and what subsequent tracks sampled this record.
- **Key Fields Rendered:**
 - *Contains Sample:* Track Title, Artist, Year, and type of sample (e.g., "Direct sample of drum break").
 - *Sampled By:* Derived tracks, artists, and year (e.g., "Sampled by J Dilla in 2006").
- **Graph Source:** Recursively queries `sample_relationships` joined with `tracks` and `releases` to traverse the tree.

Screen 4: People Graph (The Connection)

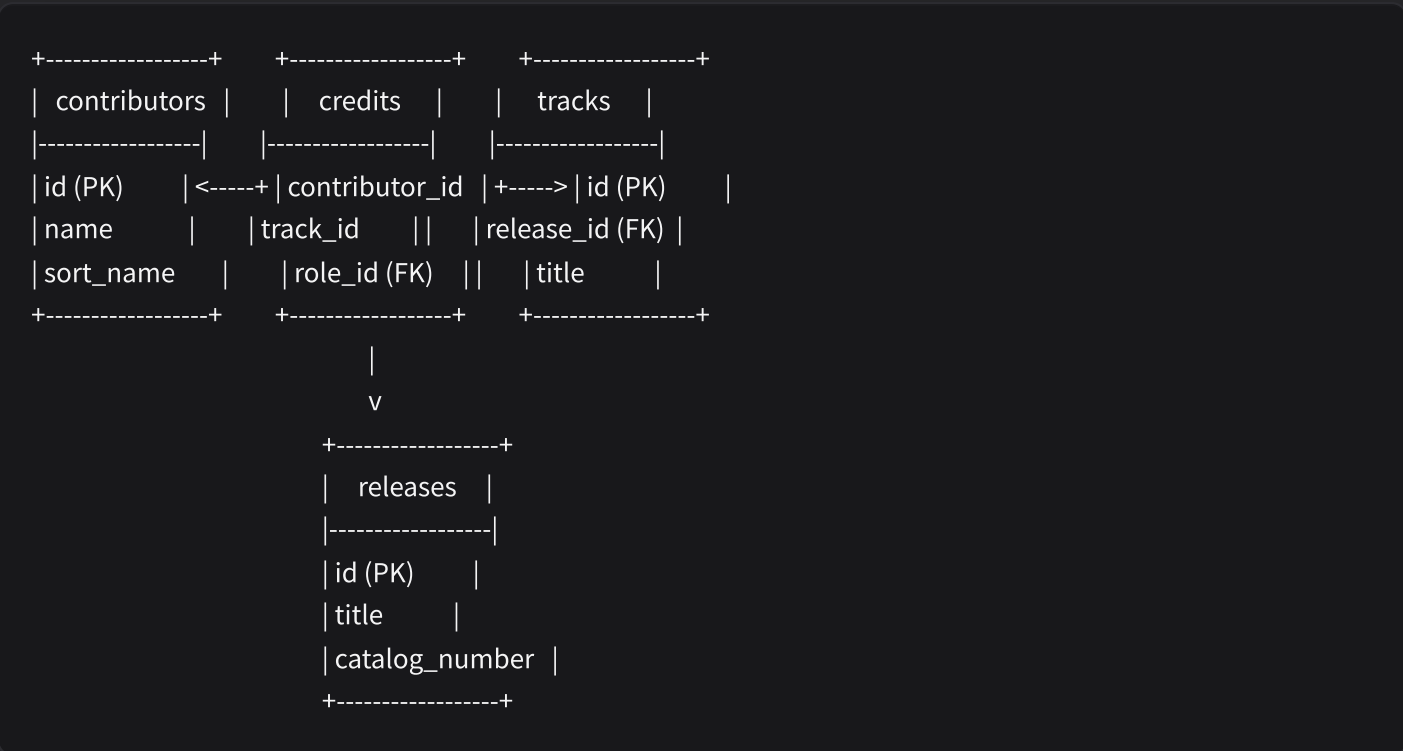
- **Purpose:** The feature that makes crate diggers "lose their minds." Clicking any musician, producer, or engineer displays their complete collaborative graph.
- **Key Fields Rendered:** Contributor Name, Role, list of other tracks in the database they performed that role on, and a list of "Related Collaborators" (who they frequently recorded with).
- **Graph Source:** Queries `credits` for a specific `contributor_id` , grouping and filtering by `role_id` .

Screen 5: Listen (The Gateway)

- **Purpose:** Launching listening endpoints once the user is ready to hear the full track or sample.
- **Key Fields Rendered:** Fast-launch `launch` links or QR codes for YouTube, TIDAL, Spotify, and Apple Music.
- **Graph Source:** Queries `external_links` associated with the active track or release.

3. SQLite Database Schema Specification

Designed for **SQLite 3** running on a bare-metal Embedded Linux kernel. Under SQLite’ s dynamic typing, we utilize strict type definitions and optimal indexing strategies to guarantee sub-100ms queries over 1M+ rows.



3.1 Table: contributors

Represents any individual, group, or entity that contributed to a recording (artists, session musicians, producers, engineers).

```
CREATE TABLE contributors (
  id TEXT PRIMARY KEY,      -- e.g., 'cont_001' or MusicBrainz UUID
  name TEXT NOT NULL,       -- Display Name: e.g., 'Roy Ayers'
  sort_name TEXT,          -- Sorting Name: e.g., 'Ayers, Roy'
  type TEXT CHECK(type IN ('person', 'group', 'orchestra', 'choir')),
  mbid TEXT UNIQUE,        -- MusicBrainz ID Link
  discogs_id INTEGER UNIQUE -- Discogs Artist ID Link
);
```

3.2 Table: releases

Represents physical vinyl releases (LPs, EPs, 7" singles, 12" promos).

```
CREATE TABLE releases (
  id TEXT PRIMARY KEY,      -- e.g., 'rel_001' or Discogs Release ID
  title TEXT NOT NULL,     -- Album Title: e.g., 'Everybody Loves The Sunshine'
  year INTEGER,            -- e.g., 1976
  label TEXT,              -- e.g., 'Polydor'
  catalog_number TEXT NOT NULL, -- e.g., 'PD-1-6070' (CRITICAL for OCR Matching)
  country TEXT,            -- e.g., 'US'
  barcode TEXT,            -- e.g., '075021321014' (From Barcode Reader)
  discogs_id INTEGER UNIQUE, -- Discogs Master ID
  mbid TEXT UNIQUE         -- MusicBrainz Release Group ID
);
```

3.3 Table: tracks

Individual songs on a physical release.

```
CREATE TABLE tracks (
  id TEXT PRIMARY KEY,      -- e.g., 'track_001'
  release_id TEXT NOT NULL, -- Foreign Key linking to releases
  title TEXT NOT NULL,     -- Song Title: e.g., 'Everybody Loves The Sunshine'
  position TEXT,           -- Physical vinyl side/position: e.g., 'A1', 'B3'
  duration_ms INTEGER,     -- Length of track in milliseconds
  FOREIGN KEY(release_id) REFERENCES releases(id) ON DELETE CASCADE
);
```

3.4 Table: roles

The master list of normalized engineering and musical roles.

```
CREATE TABLE roles (
  id TEXT PRIMARY KEY,      -- e.g., 'role_bass', 'role_producer', 'role_engineer'
  name TEXT NOT NULL,       -- e.g., 'Double Bass', 'Producer', 'Mixing Engineer'
);
```

```
role_group TEXT NOT NULL CHECK(role_group IN ('musician', 'production', 'engineering', 'vocals', 'visual'))
);
```

3.5 Table: `credits` (The Core Edge)

The critical relationship edge connecting contributors to tracks via their specific roles. This is the single highest-frequency query table in `yadiggg`.

```
CREATE TABLE credits (
  track_id TEXT NOT NULL,
  contributor_id TEXT NOT NULL,
  role_id TEXT NOT NULL,
  source TEXT NOT NULL,      -- e.g., 'MusicBrainz', 'Discogs', 'TIDAL', 'Manual'
  confidence REAL DEFAULT 1.0, -- Match confidence score (0.0 to 1.0)
  PRIMARY KEY(track_id, contributor_id, role_id),
  FOREIGN KEY(track_id) REFERENCES tracks(id) ON DELETE CASCADE,
  FOREIGN KEY(contributor_id) REFERENCES contributors(id) ON DELETE CASCADE,
  FOREIGN KEY(role_id) REFERENCES roles(id) ON DELETE RESTRICT
);
```

3.6 Table: `sample_relationships` (The Lineage Edge)

A directed graph capturing sample, interpolation, cover, and remix relationships between tracks.

```
CREATE TABLE sample_relationships (
  id TEXT PRIMARY KEY,
  source_track_id TEXT NOT NULL, -- The track that CONTAINS/USES the sample (e.g., track_900)
  sampled_track_id TEXT NOT NULL, -- The original track that WAS SAMPLED (e.g., track_001)
  type TEXT NOT NULL CHECK(type IN ('direct_sample', 'interpolation', 'cover', 'remix')),
  description TEXT,              -- e.g., 'drum break loop', 'vocal hook melody', 'bassline'
  confidence REAL DEFAULT 1.0,   -- WhoSampled / crowdsourced confidence
  FOREIGN KEY(source_track_id) REFERENCES tracks(id) ON DELETE CASCADE,
  FOREIGN KEY(sampled_track_id) REFERENCES tracks(id) ON DELETE CASCADE
);
```

3.7 Table: `recording_locations`

Represents the recording studio, room, or venue where the track or album was cut.

```
CREATE TABLE recording_locations (
  id TEXT PRIMARY KEY,      -- e.g., 'loc_001'
  name TEXT NOT NULL,       -- e.g., 'Stankonia Studios'
  city TEXT,                -- e.g., 'Atlanta'
  state TEXT,               -- e.g., 'GA'
  country TEXT              -- e.g., 'US'
);
```

3.8 Table: `track_locations`

Links tracks to the studios where they were tracked, mixed, or mastered.

```
CREATE TABLE track_locations (  
  track_id TEXT NOT NULL,  
  location_id TEXT NOT NULL,  
  PRIMARY KEY(track_id, location_id),  
  FOREIGN KEY(track_id) REFERENCES tracks(id) ON DELETE CASCADE,  
  FOREIGN KEY(location_id) REFERENCES recording_locations(id) ON DELETE CASCADE  
);
```

3.9 Table: `external_links`

Deep-launch URLs for listening endpoints across platforms.

```
CREATE TABLE external_links (  
  id TEXT PRIMARY KEY,  
  track_id TEXT,           -- Option 1: Link is for a specific song  
  release_id TEXT,         -- Option 2: Link is for a whole album  
  platform TEXT NOT NULL CHECK(platform IN ('youtube', 'tidal', 'spotify', 'apple_music')),  
  url TEXT NOT NULL,       -- Deep-link URI  
  FOREIGN KEY(track_id) REFERENCES tracks(id) ON DELETE CASCADE,  
  FOREIGN KEY(release_id) REFERENCES releases(id) ON DELETE CASCADE  
);
```

3.10 Table: `wishlist`

The user's local "saved digs" and wishlist folder.

```
CREATE TABLE wishlist (  
  id TEXT PRIMARY KEY,  
  release_id TEXT,         -- The saved album  
  track_id TEXT,          -- The specific track (optional)  
  scanned_at TEXT NOT NULL, -- ISO 8601 Timestamp: e.g., '2026-06-03T04:08:51Z'  
  notes TEXT,             -- Custom user memo  
  audio_preview_path TEXT, -- Local absolute path to the 30s OGG/MP3 cached file  
  FOREIGN KEY(release_id) REFERENCES releases(id) ON DELETE SET NULL,  
  FOREIGN KEY(track_id) REFERENCES tracks(id) ON DELETE SET NULL  
);
```

4. Performance Indexing & Query Optimizations

To maintain a sub-100ms hardware query speed on the Cortex-A53 CPU when executing joins across tables with 1,000,000+ entries, strict index mappings are compiled on the SQLite database:

4.1 Index Map

```

-- 1. Index on catalog numbers for lightning-fast OCR matches
CREATE INDEX idx_releases_catalog ON releases(catalog_number);

-- 2. Index on track-release relationship for fast album tracklist loads
CREATE INDEX idx_tracks_release ON tracks(release_id);

-- 3. Core Credit Joins (Crucial for deep credits and contributor lookups)
CREATE INDEX idx_credits_track ON credits(track_id);
CREATE INDEX idx_credits_contributor ON credits(contributor_id);

-- 4. Sample Graph Traversal (Traversing WhoSampled-style lines)
CREATE INDEX idx_samples_source ON sample_relationships(source_track_id);
CREATE INDEX idx_samples_sampled ON sample_relationships(sampled_track_id);

-- 5. Barcode index for immediate matching
CREATE INDEX idx_releases_barcode ON releases(barcode);

```

4.2 High-Leverage Query Examples

Query for Screen 2: Deep Credits Joins

Retrieves session musicians, engineers, and producers for a given track:

```

SELECT
  c.name AS contributor_name,
  r.name AS credit_role,
  r.role_group AS role_category
FROM credits cr
JOIN contributors c ON cr.contributor_id = c.id
JOIN roles r ON cr.role_id = r.id
WHERE cr.track_id = 'track_001'
ORDER BY r.role_group, c.name;

```

Query for Screen 4: Traversing the Connection Graph (People Graph)

Displays every track where William Allen played bass in the database, showing associated release year and label details:

```

SELECT
  t.title AS track_title,
  rel.title AS album_title,
  rel.year AS release_year,
  rel.label AS record_label
FROM credits cr
JOIN tracks t ON cr.track_id = t.id
JOIN releases rel ON t.release_id = rel.id
WHERE cr.contributor_id = 'person_william_allen'

```

```
AND cr.role_id = 'role_bass'  
ORDER BY rel.year DESC;
```